Revisão: linguagens formais e compiladores

Mapa da revisão

Nesta revisão, vamos conectar os conceitos de linguagens formais com exemplos práticos de reconhecimento e geração de palavras. A sequência parte de alfabetos e expressões regulares, avança para autômatos e conversões e termina com gramáticas, árvores sintáticas e normalização.



Alfabeto, palavra e linguagem

Conceitos fundamentais

Alfabeto é um conjunto finito de símbolos, representado por Σ. Uma palavra é uma sequência finita desses símbolos, e linguagem é um conjunto de palavras. Se Σ = {a,b}, então abba é uma palavra possível. A palavra vazia ε tem comprimento zero.

Σ = {a,b}

w = abba

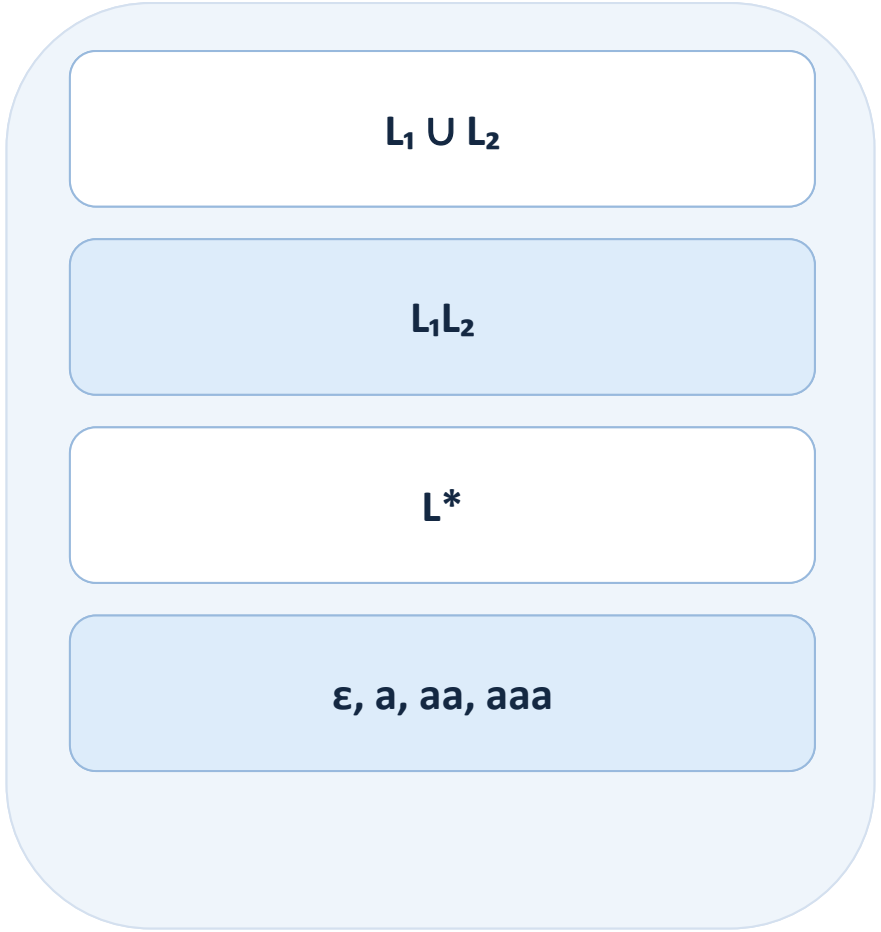
ε = vazio

L ⊆ Σ*

Operações com linguagens

União, concatenação e repetição

A união reúne palavras de duas linguagens. A concatenação forma novas palavras colocando uma depois da outra. O fecho de Kleene, indicado por asterisco, permite zero ou mais repetições. Por exemplo, se L = {a}, então L* contém ε, a, aa e aaa.



Hierarquia de Chomsky

Classes de linguagens

A hierarquia organiza gramáticas conforme as restrições das produções e o poder de reconhecimento. O Tipo 3 representa linguagens regulares; o Tipo 2, livres de contexto; o Tipo 1, sensíveis ao contexto; e o Tipo 0, irrestritas. Cada classe contém as classes mais restritas.



Qual modelo utilizar?

Comparação por exemplos

Um padrão de identificador simples pode ser descrito por expressão regular e reconhecido por autômato finito. Já a linguagem $a^n b^n$ precisa conferir a mesma quantidade de a e b, exigindo memória de pilha. A linguagem $a^n b^n c^n$ exige dependência entre três blocos e é sensível ao contexto.

Identificador → Tipo 3

$a^n b^n$ → Tipo 2

$a^n b^n c^n$ → Tipo 1

Gramáticas regulares

Produções à direita

Uma gramática regular à direita usa produções como S → aA, A → bA e A → ε. A primeira regra gera a letra a, a segunda permite repetir b e a última encerra a derivação. Assim, essa gramática gera a linguagem descrita pela expressão regular ab*.

S → aA

A → bA

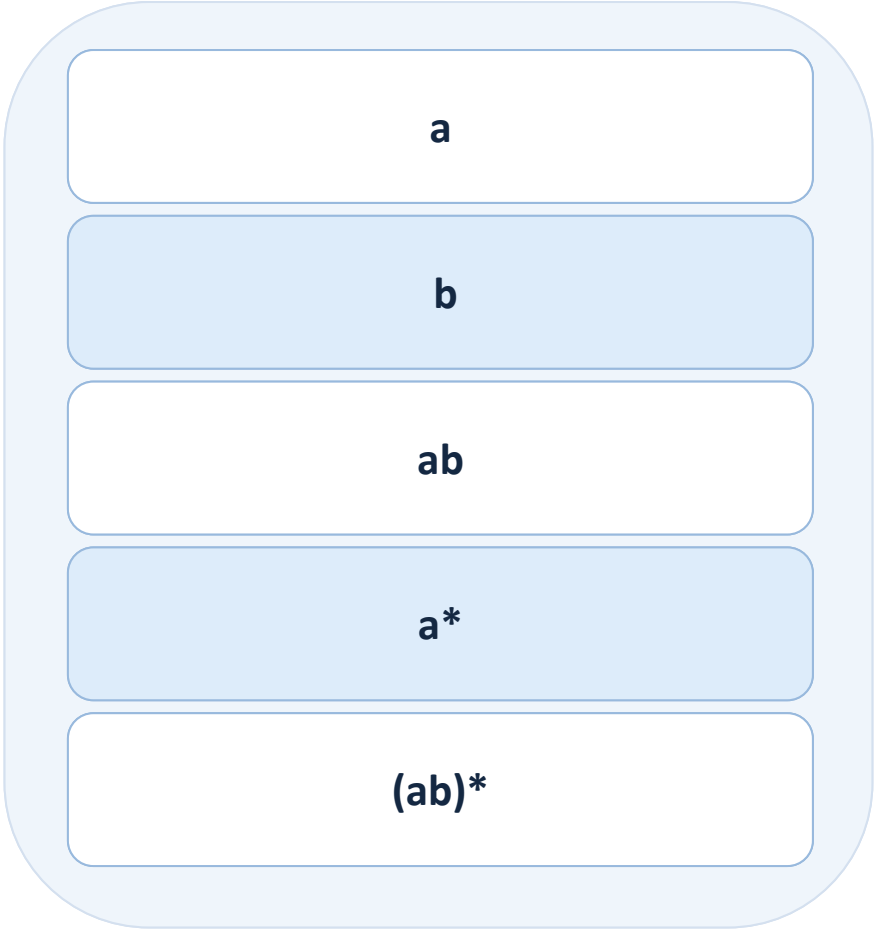
A → ε

ER: ab*

Expressões regulares

Operadores e prioridade

Expressões regulares descrevem padrões por meio de símbolos, alternativas, concatenação e repetição. A expressão a|b aceita a ou b; ab exige a seguido de b; a* permite zero ou mais letras a. Os parênteses determinam o agrupamento e evitam interpretações incorretas.



Não confunda ab* com (ab)*

Interpretação de padrões

Em ab*, a letra a é obrigatória e pode ser seguida de qualquer quantidade de b: a, ab e abb. Em (ab)*, o bloco inteiro ab é repetido zero ou mais vezes: ε, ab e abab. A posição dos parênteses muda a linguagem reconhecida.

ab* → a, ab, abb

(ab)* → ε, ab, abab

Regex para identificadores

Exemplo de análise léxica

Um identificador simplificado deve começar por uma letra e depois pode conter zero ou mais letras ou dígitos. A expressão letra(letra|digito)* representa essa regra. Ela aceita x, total2 e valor10, mas rejeita 2valor porque o primeiro símbolo não é uma letra.



Autômato finito: cinco componentes

$M = (Q, \Sigma, \delta, q_0, F)$

Um autômato finito possui estados Q, alfabeto Σ, função ou relação de transição δ, estado inicial q_0 e estados finais F. Ele consome uma palavra símbolo por símbolo. Ao terminar a entrada, verifica se a execução chegou a um estado final, representado por círculo duplo.

Q: estados

Σ: símbolos

δ: transições

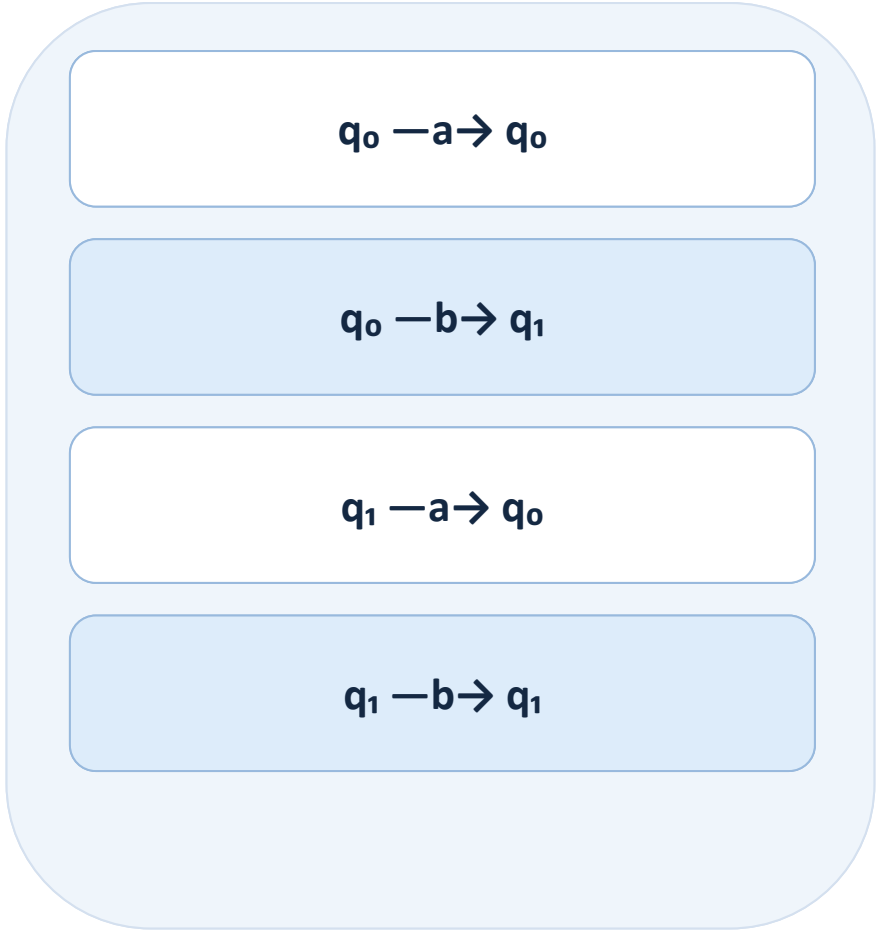
q_0: início

F: finais

AFD: um caminho por entrada

Autômato determinístico

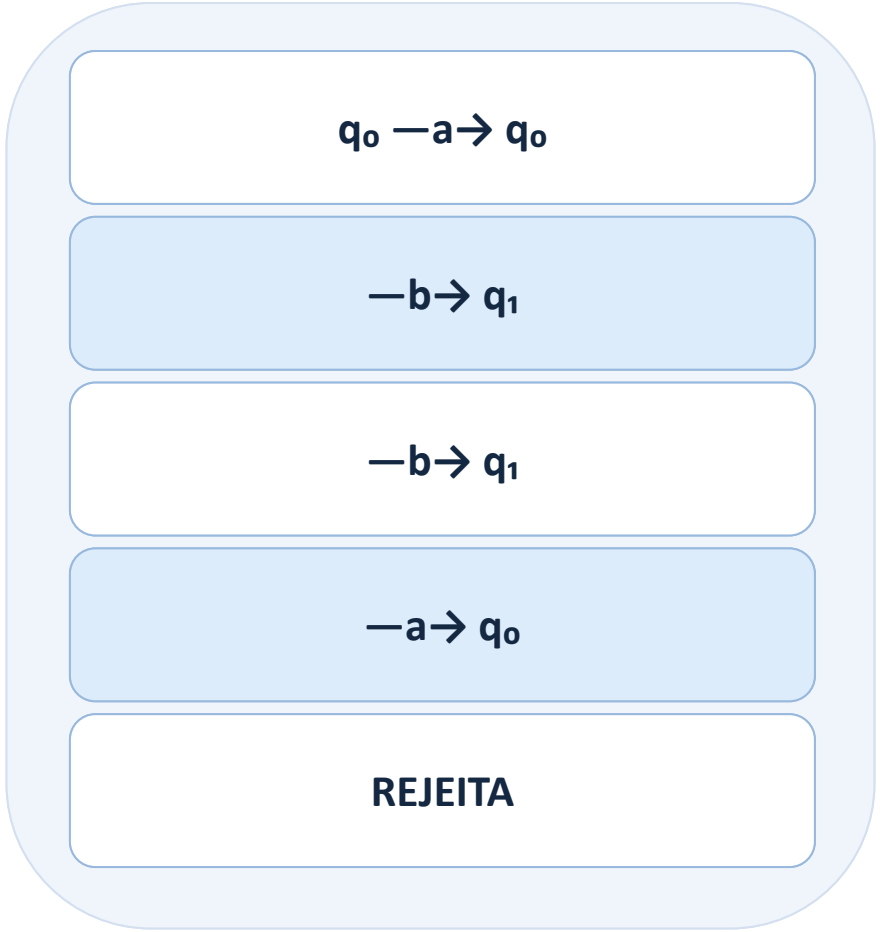
Em um AFD, cada par estado e símbolo determina um único próximo estado. Em um AFD completo, todas as combinações possuem transição; um estado morto pode receber entradas que não levam à aceitação. O reconhecimento ocorre por uma execução única, acompanhada desde o estado inicial.



Simulação de AFD: termina em b

Exercício resolvido

Considere q_0 inicial e não final, e q_1 final. A leitura de a sempre leva a q_0, enquanto b sempre leva a q_1. Para a entrada abba, os estados visitados são q_0, q_0, q_1, q_1 e q_0. Como a entrada termina em q_0, a palavra é rejeitada.



AFD: quantidade par de 1

Construção e simulação

Para reconhecer palavras binárias com quantidade par de símbolos 1, use q_0 como estado inicial e final, e q_1 como estado ímpar. Ler 0 mantém o estado; ler 1 alterna entre os dois. A palavra 1011 possui três símbolos 1 e termina em q_1, portanto é rejeitada.

q_0: par (final)

q_1: ímpar

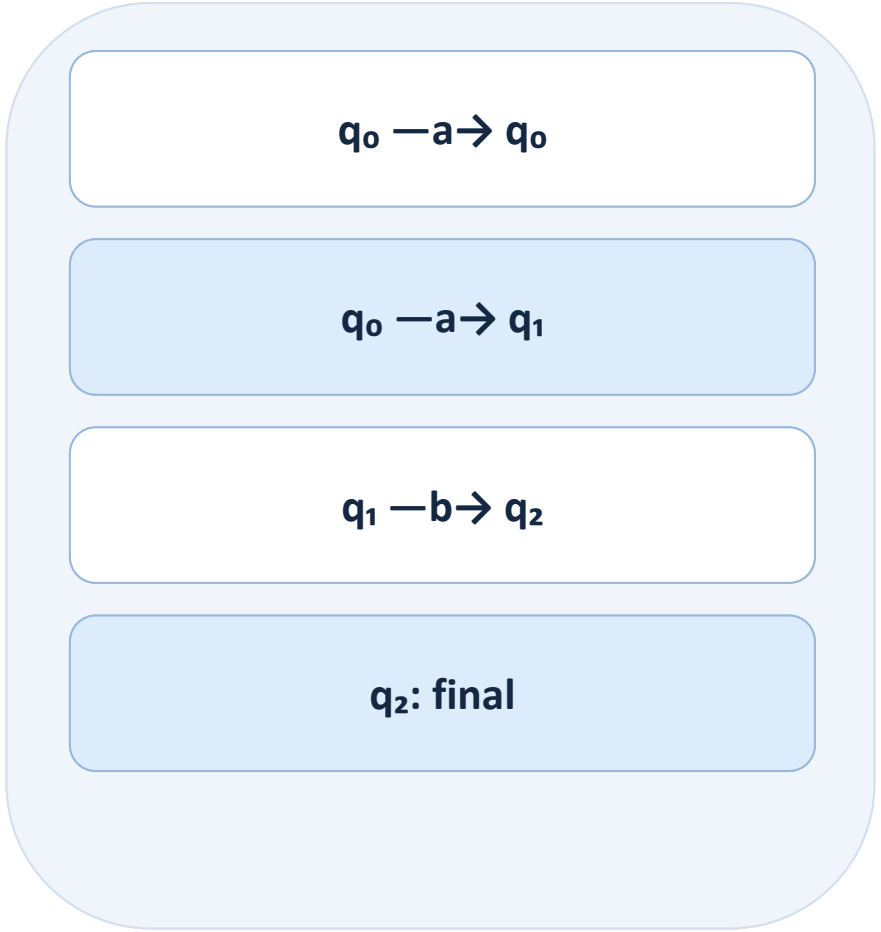
0 → permanece

1 → alterna

AFN: diferentes caminhos

Não determinismo e aceitação

Um AFN pode oferecer mais de uma transição para o mesmo símbolo. Isso significa que vários estados podem ser possíveis durante a leitura. A palavra é aceita se existir pelo menos um caminho que consuma toda a entrada e termine em estado final. Não é necessário que todos os caminhos aceitem.



Transições vazias e ε-fecho

Mover sem consumir símbolos

Uma transição ε permite mudar de estado sem ler um símbolo da palavra. O ε-fecho de um estado reúne esse estado e todos os estados alcançáveis apenas por transições vazias. Antes de processar uma entrada em um AFN com ε, precisamos considerar todos os estados desse conjunto.

$q_0 \xrightarrow{\varepsilon} q_1$

$q_1 \xrightarrow{a} q_2$

$\varepsilon\text{-fecho}(q_0)=\{q_0,q_1\}$

AFN e AFD reconhecem o mesmo

Equivalência de poder

AFNs e AFDs reconhecem exatamente as linguagens regulares. O AFN pode facilitar a descrição de um padrão porque oferece escolhas e transições vazias. Entretanto, é possível construir um AFD equivalente, no qual cada estado representa um conjunto de estados possíveis do AFN, sem alterar as palavras aceitas.



Regex → AFN

Construção por blocos

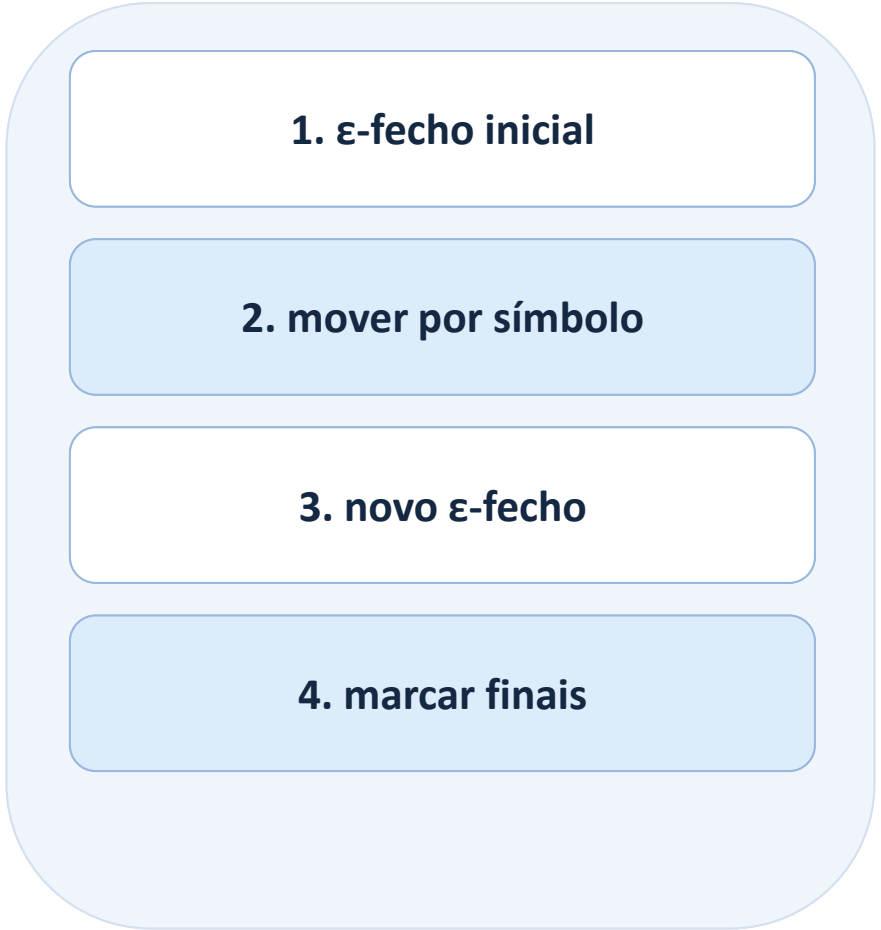
Para transformar uma expressão regular em AFN, construímos blocos para símbolos simples e depois combinamos os blocos. A união oferece caminhos alternativos; a concatenação liga blocos em sequência; o fecho de Kleene cria possibilidade de repetição e de zero ocorrências. As transições ε ajudam a conectar as partes.



AFN → AFD: passo a passo

Construção por subconjuntos

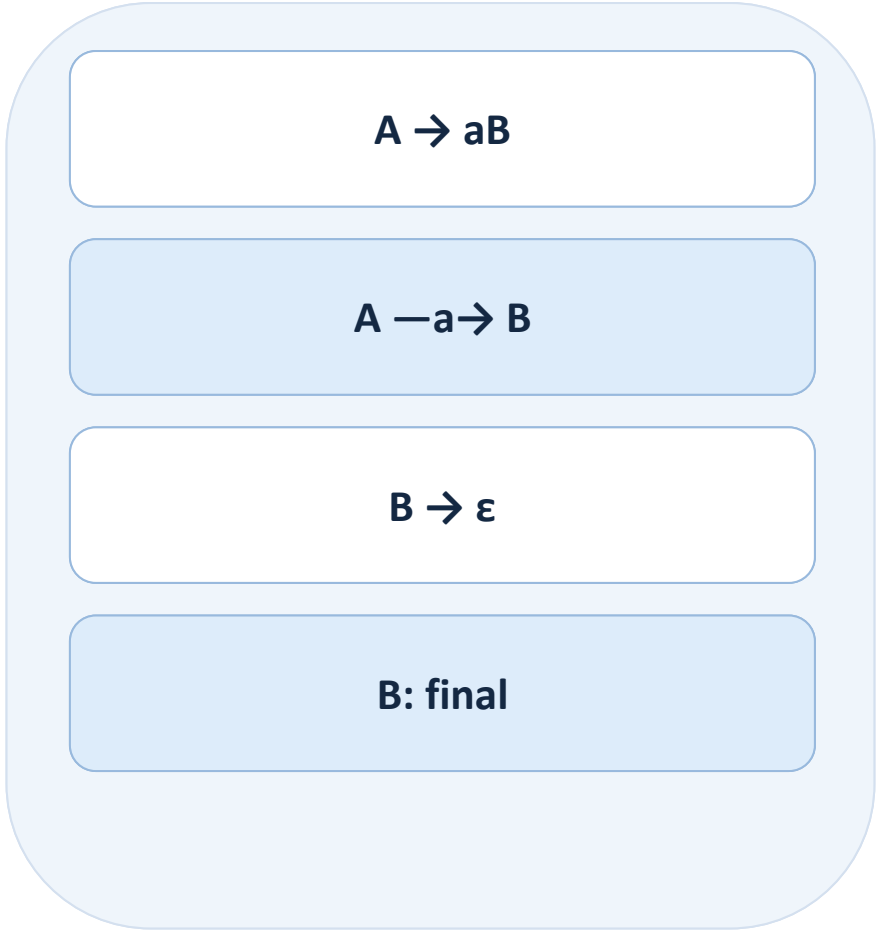
Comece calculando o ε-fecho do estado inicial do AFN. Esse conjunto será o estado inicial do AFD. Para cada símbolo, reúna todos os destinos possíveis e aplique novamente o ε-fecho. Repita até não surgirem novos conjuntos. Um conjunto é final quando contém algum estado final do AFN.



Gramática regular → autômato

Correspondência entre modelos

Na conversão de uma gramática regular à direita, cada variável pode originar um estado. Uma produção A → aB cria uma transição com a do estado A para B. Uma produção B → ε indica que B pode ser final. O estado associado ao símbolo inicial da gramática será o estado inicial do autômato.



Minimização de AFD

Redução sem alterar a linguagem

Minimizar um AFD significa reduzir a quantidade de estados preservando exatamente a linguagem reconhecida. Primeiro, removemos estados inalcançáveis. Depois, separamos estados finais e não finais e refinamos os grupos conforme suas transições. Estados que continuam indistinguíveis podem ser reunidos em um único estado.



GLC: definição e componentes

G = (V, T, S, P)

Uma gramática livre de contexto é formada por variáveis V, terminais T, símbolo inicial S e produções P. Cada produção substitui uma única variável, independentemente dos símbolos ao redor. Essa característica permite descrever estruturas aninhadas, como parênteses balanceados e expressões de linguagens de programação.

V: variáveis

T: terminais

S: inicial

P: regras

Derivação de $a^n b^n$

S → aSb | ε

A gramática S → aSb | ε gera a mesma quantidade de letras a e b, com todos os a antes dos b. Para gerar aabb, aplicamos S → aSb duas vezes e depois substituímos S por ε. As sequências intermediárias são chamadas formas sentenciais.

S ⇒ aSb

⇒ aaSbb

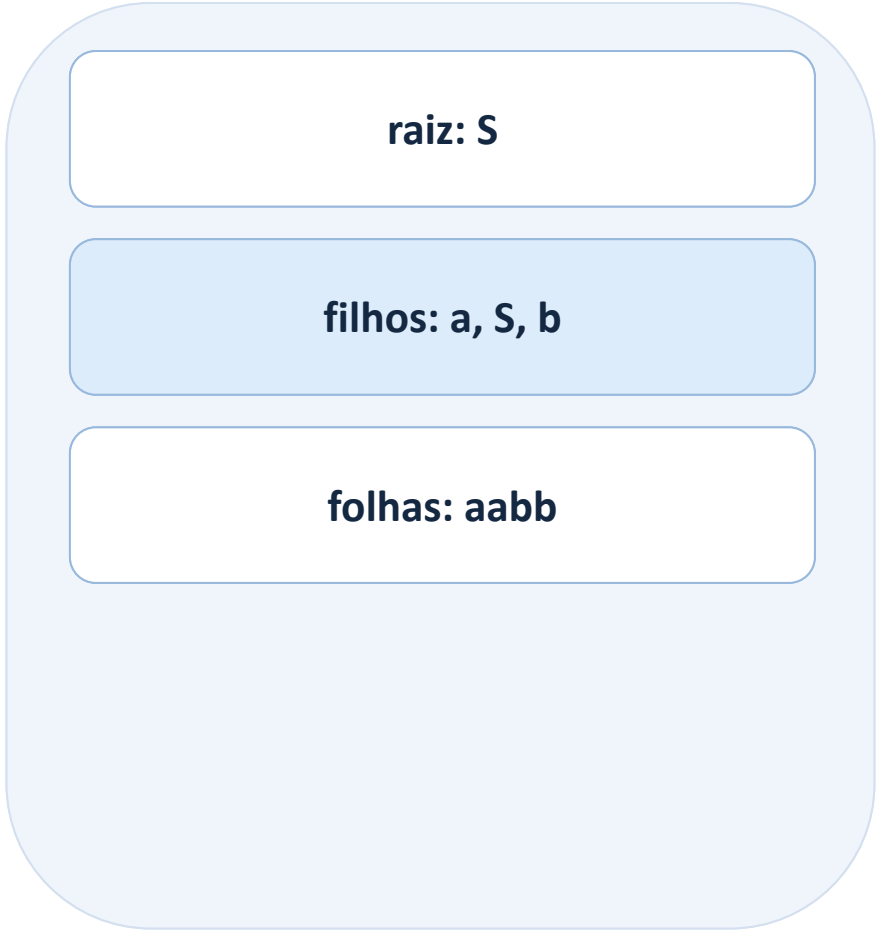
⇒ aabb

n = 2

Derivações e árvores sintáticas

Duas maneiras de representar

Uma derivação registra a sequência de produções aplicadas à gramática. A árvore sintática organiza as mesmas escolhas de maneira hierárquica: a raiz é o símbolo inicial, os nós internos representam variáveis e as folhas formam a palavra final. Derivações à esquerda e à direita diferem pela ordem de expansão.



Ambiguidade de gramáticas

Mais de uma árvore possível

Uma gramática é ambígua quando uma mesma sentença pode ter duas árvores sintáticas diferentes. Na gramática E → E + E | E * E | id, a expressão id + id * id pode representar (id + id) * id ou id + (id * id). A interpretação depende da estrutura escolhida.

(id + id) * id

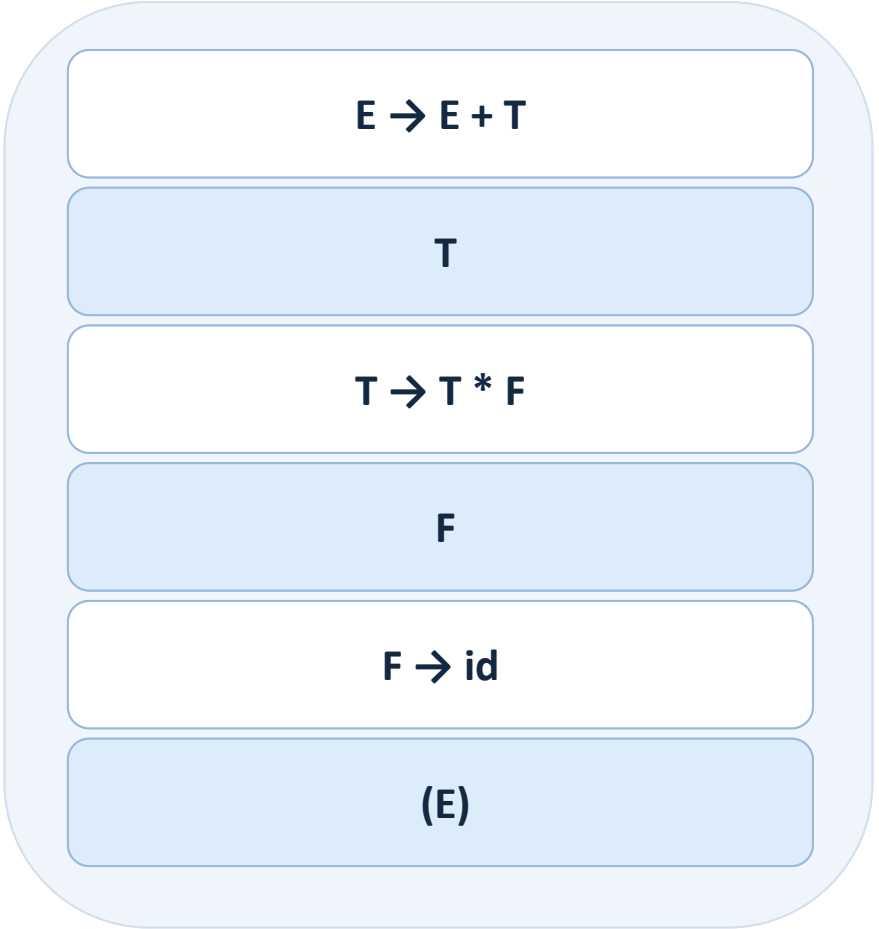
id + (id * id)

duas árvores

Precedência e associatividade

Organizando expressões

Para retirar a ambiguidade usual de expressões aritméticas, podemos usar níveis E, T e F. A regra E → E + T | T trata soma; T → T * F | F trata multiplicação; F → id | (E) trata operandos. A recursão à esquerda estabelece associação à esquerda.



Forma Normal de Chomsky

Formatos de produções

Na Forma Normal de Chomsky, as produções usuais seguem A → BC ou A → a, com variáveis no primeiro formato e um terminal no segundo. Se a linguagem contiver a palavra vazia, admite-se uma exceção controlada para o novo símbolo inicial. A normalização padroniza regras, mas não garante ausência de ambiguidade.

A → BC

A → a

S_0 → ε (se cabível)

FNC ≠ desambiguação

Como preparar uma GLC para FNC

Sequência de transformação

Primeiro, criamos um novo símbolo inicial quando necessário. Em seguida, eliminamos produções vazias, exceto a exceção permitida, e produções unitárias. Removemos símbolos inúteis. Por fim, substituímos terminais em produções longas e dividimos lados direitos com mais de duas variáveis em regras binárias.



Prática gratuita com JFLAP

Construção e simulação

No JFLAP, escolha Finite Automaton e crie dois estados para reconhecer palavras terminadas em b. Marque q_0 como inicial e q_1 como final. Adicione as quatro transições para a e b. Use Input e a opção de simulação para testar ab, abba e b, verificando os resultados.

1. Criar estados
2. Marcar início/final
3. Inserir transições
4. Testar palavras

Atividade de revisão

Aplicação integrada

Em duplas, descrevam uma expressão regular para palavras binárias terminadas em 01 e construam um AFD equivalente. Depois, simulem as entradas 01, 101 e 010. Justifiquem quais palavras são aceitas com base no estado alcançado após o consumo completo da entrada.

ER: (0

1)*01

01 ✓

101 ✓

010 ✗

Avaliação da revisão e referências

Síntese e materiais de apoio

Confira se você consegue interpretar expressões regulares, simular autômatos, explicar o ε-fecho, converter modelos, minimizar AFDs, derivar palavras e reconhecer ambiguidade. Para aprofundar, consulte os slides da disciplina, o JFLAP e livros introdutórios de linguagens formais e compiladores.

JFLAP: jflap.org

Sipser: Computação

Aho et al.: Compiladores

Slides da disciplina